4xx Client Error HTTP Status Codes (400, 401, 403, 404, 405)

Meaning: There was an error on the client's side — bad syntax, invalid data, missing authentication, or requesting a non-existent resource.

Category: 4xx (Client Error)

Key Point: The problem is with the CLIENT's request, not the server.

4xx Status Codes

Status Code 400:

- **Name:** Bad Request
- **Meaning:** Server cannot process due to client error
- **When to Use:** Missing required field, invalid data type, validation failure
- **ASP.NET Core Helper:** BadRequest()

Status Code 401:

- **Name:** Unauthorized
- **Meaning:** Authentication required, failed, or not provided
- **When to Use:** Missing API key, invalid token, expired credentials
- **ASP.NET Core Helper:** Unauthorized()

Status Code 403:

- **Name:** Forbidden
- **Meaning:** Authentication but not authorized to access resources
- **When to Use:** Valid user but no permission (regular user accessing admin endpoint)
- **ASP.NET Core Helper:** Forbid()

Status Code 404:

- **Name:** Not Found
- **Meaning:** Required resource does not exist
- **When to Use:** Wrong URL, deleted resource, invalid ID

- **ASP.NET Core Helper:** Not Found()

Status Code 405:

- **Name:** Method Not Allowed
- **Meaning:** HTTP method not supported
- **When to Use:** POST endpoint called wit GET verb
- **ASP.NET Core Helper:** Automatic (framework)

Now we are going to set up a project ‘ClothsApi3_ClientErrorStatusCodes’ to represent the above concepts,

Project Setup

<u>Create the project:</u>

Command: dotnet new webapi -n ClothsApi3_ClientErrorStatusCodes -f net8.0

<u>Navigate to the project:</u>

Command: cd ClothsApi3_ClientErrorStatusCodes

Install Packages

Before installing the packages, we should make sure that we are in the project folder ‘ClothsApi3_ClientErrorStatusCodes’ in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

ClothsApi3_ClientErrorStatusCodes.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Create Cloth Model

Path: ClothsApi3_ClientErrorStatusCodes/Models/Cloth.cs

Cloth.cs

Models > Cloth.cs > ...

```
namespace ClothsApi3_ClientErrorStatusCodes.Models
{
    0 references
    public class Cloth
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Name { get; set; } = string.Empty;
        0 references
        public string Category { get; set; } = string.Empty;
        0 references
        public string Size { get; set; } = string.Empty;
        0 references
        public string Color { get; set; } = string.Empty;
        0 references
        public decimal Price { get; set; }
        0 references
        public bool InStock { get; set; }
    }
}
```

Step 2: Create Cloth Service

Path: ClothsApi3_ClientErrorStatusCodes/Services/ClothService.cs

ClothService.cs

Services > ClothService.cs > ...

```
using ClothsApi3_ClientErrorStatusCodes.Models;

namespace ClothsApi3_ClientErrorStatusCodes.Services
{
    0 references
    public static class ClothService
    {
        // In-memory data (7 cloth items)
        4 references
        private static readonly List<Cloth> _cloths = new()
        {
            new Cloth {
                Id = 1,
                Name = "Cotton T-Shirt",
                Category = "Men",
                Size = "M",
                Color = "Blue",
                Price = 599,
                InStock = true ,
            },
            new Cloth {
                Id = 2,
                Name = "Denim Jacket",
                Category = "Women",
                Size = "L",
                Color = "Black",
                Price = 2499,
                InStock = true ,
            },
            new Cloth {
                Id = 3,
                Name = "Formal Shirt",
                Category = "Men",
                Size = "XL",
                Color = "White",
                Price = 1299,
                InStock = false,
            },
            new Cloth {
                Id = 4,
                Name = "Sports Shoes",
                Category = "Men",
                Size = "9",
                Color = "Grey",
                Price = 3499,
                InStock = true,
            },
            new Cloth {
                Id = 5,
                Name = "Silk Saree",
                Category = "Women",
                Size = "Free",
                Color = "Red",
                Price = 4999,
                InStock = true,
            },
            new Cloth {
                Id = 6,
                Name = "Kids Hoodie",
                Category = "Kids",
                Size = "S",
                Color = "Yellow",
                Price = 899,
                InStock = true,
            },
            new Cloth {
                Id = 7,
                Name = "Leather Belt",
                Category = "Accessories",
                Size = "32",
                Color = "Brown",
                Price = 399,
                InStock = false,
            }
        };

        // Get all cloths
        0 references
        public static async Task<List<Cloth>> GetAllClothsAsync()
        {
            await Task.Delay(50);
            return _cloths;
        }
```

```
        // Get cloth by ID
        0 references
        public static async Task<Cloth?> GetClothByIdAsync(int id)
        {
            await Task.Delay(30);
            return _cloths.FirstOrDefault(c => c.Id == id);
        }
```

```
        // Add new cloth
        0 references
        public static async Task<Cloth> AddClothAsync(Cloth newCloth)
        {
            await Task.Delay(100);
            newCloth.Id = _cloths.Max(c => c.Id) + 1;
            _cloths.Add(newCloth);
            return newCloth;
        }

        // Simulate user authentication
        0 references
        public static bool IsValidApiKey(string? apiKey)
        {
            // Valid API key is "valid-key-123"
            return apiKey == "valid-key-123";
        }

        // Simulate admin role check
        0 references
        public static bool IsAdminUser(string? role)
        {
            // Admin role required
            return role == "admin";
        }
    }
}
```

Step 3: Program.cs (Minimal)

Path: ClothsApi3_ClientErrorStatusCodes/Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Step 4: Controller (All 4xx Status Codes)

Path: ClothsApi3_ClientErrorStatusCodes/Controllers/ClothController.cs

ClothController.cs

Controllers > ClothController.cs > ...

```
using ClothsApi3_ClientErrorStatusCodes.Models;
using ClothsApi3_ClientErrorStatusCodes.Services;
using Microsoft.AspNetCore.Mvc;

namespace ClothsApi3_ClientErrorStatusCodes.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    0 references
    public class ClothController : ControllerBase
    {
        // ========================================================
        // 400 Bad Request -- Invalid input from client
        // Helper: BadRequest()
        // ========================================================

        // POST /api/Cloth
        // Returns 400 if Name is missing
        [HttpPost]
        0 references
        public async Task<ActionResult<Cloth>> CreateCloth(
            [FromBody] Cloth newCloth
        )
        {
            // 400: Name is required
            if (string.IsNullOrWhiteSpace(newCloth.Name))
                return BadRequest("Cloth name is required. Please Provide a valid name.");

            // 400: Price must be greater than 0
            if (newCloth.Price <= 0)
                return BadRequest("Price must be greater than zero.");

            // 400: Category is required
            if (string.IsNullOrWhiteSpace(newCloth.Category))
                return BadRequest("Category is required. (Men, Women, Kids, Accessories)");

            var createdCloth = await ClothService.AddClothAsync(newCloth);

            return CreatedAtAction(
                nameof(GetClothById),
                new {id = createdCloth.Id},
                createdCloth
            );
        }

```

```
44        // ==========================================================
45        // 401 Unauthorized -- Authentication failed or not provided
46        // Helper: Unauthorized()
47        // ==========================================================
48
49        // GET /api/Cloth/admin/all
50        // Returns 401 if API key is missing or invalid
51        [HttpGet("admin/all")]
          0 references
52        public async Task<ActionResult<List<Cloth>>> GetAdminAllCloths(
53            [FromHeader(Name = "X-API-Key")] string? apiKey
54        )
55        {
56            // 401: API key is missing
57            if (string.IsNullOrEmpty(apiKey))
58                return Unauthorized("API key is missing. Please provide a valid API key.");
59
60            // 401: API Key is invalid
61            if (!ClothService.IsValidApiKey(apiKey))
62                return Unauthorized("Invalid API key. Authentication failed.");
63
64            var cloths = await ClothService.GetAllClothsAsync();
65            return Ok(cloths);
66        }
67
```

```
68        // ==========================================================
69        // 403 Forbidden -- Authenticated but not authorized
70        // Helper: Forbid()
71        // ==========================================================
72
73        // GET /api/Cloth/admin/delete/{id}
74        // Returns 401 if no API key, 403 if API key valid but not admin role
75        [HttpGet("admin/delete/{id}")]
          0 references
76        public async Task<IActionResult> DeleteClothAdmin(
77            int id,
78            [FromHeader(Name ="X-API-Key")] string? apiKey,
79            [FromHeader(Name ="X-User-Role")] string? userRole
80        )
81        {
82            // 401: API key is missing
83            if (string.IsNullOrEmpty(apiKey))
84                return Unauthorized("API key is missing. Please provide a valid API key.");
85
86            // 401: API key is invalid
87            if (!ClothService.IsValidApiKey(apiKey))
88                return Unauthorized("Invalid API key. Authentication failed.");
89
90            // 403: Valid API key but not admin role
91            if (!ClothService.IsAdminUser(userRole))
92                return Forbid("Only admin users can delete products.");
93
94            var cloth = await ClothService.GetClothByIdAsync(id);
95            if(cloth == null)
96                return NotFound($"Cloth with ID {id} not found");
97
98            return Ok($"Cloth '{cloth.Name}' deleted successfully (simulated)");
99        }
```

```
        // ============================================================
        // 404 Not Found -- Resource does not exist
        // Helper: NotFound()
        // ============================================================

        // GET /api/Cloth/{id}
        // Returns 404 if cloth with given ID does not exist
        [HttpGet("{id}")]
        1 reference
        public async Task<ActionResult<Cloth>> GetClothById(int id)
        {
            var cloth = await ClothService.GetClothByIdAsync(id);

            // 404: Cloth not found
            if (cloth == null)
                return NotFound($"Cloth with ID {id} was not found.");

            return Ok(cloth);
        }

        // GET /api/Cloth (all cloths -- works fine)
        [HttpGet]
        0 references
        public async Task<ActionResult<List<Cloth>>> GetAllCloths()
        {
            var cloths = await ClothService.GetAllClothsAsync();
            return Ok(cloths);
        }

        // ============================================================
        // 405 Method Not Allowed -- HTTP method not supported
        // Handled automatically by framework
        // ============================================================

        // Note: This endpoint only supports GET
        // If you send POST request to /api/Cloth/5, you will get 405
        [HttpGet("{id}/details")]
        0 references
        public async Task<ActionResult<Cloth>> GetClothDetails(int id)
        {
            var cloth = await ClothService.GetClothByIdAsync(id);
            if (cloth == null)
                return NotFound($"Cloth with ID {id} not found");

            return Ok(cloth);
        }
    }
}
```

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (ClothsApi3_ClientErrorStatusCodes) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi3_ClientErrorStatusCodes> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  ClothsApi3_ClientErrorStatusCodes -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi3_Clie
  ntErrorStatusCodes\bin\Debug\net8.0\ClothsApi3_ClientErrorStatusCodes.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:01.11
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi3_ClientErrorStatusCodes> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dotnet/hot-reload.
   💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  ClothsApi3_ClientErrorStatusCodes -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi3_Clie
  ntErrorStatusCodes\bin\Debug\net8.0\ClothsApi3_ClientErrorStatusCodes.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5185
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi3_ClientErrorStatusCo
des
```

ClothsApi3_ClientErrorStatusCodes 1.0 OAS 3.0

http://localhost:5185/swagger/v1/swagger.json

Cloth

POST	/api/Cloth
GET	/api/Cloth
GET	/api/Cloth/admin/all
GET	/api/Cloth/admin/delete/{id}
GET	/api/Cloth/{id}
GET	/api/Cloth/{id}/details

Schemas

Cloth >

We are going to use **Postman** here to test our APIs,

Test 1: 200 OK – Get Single Cloth

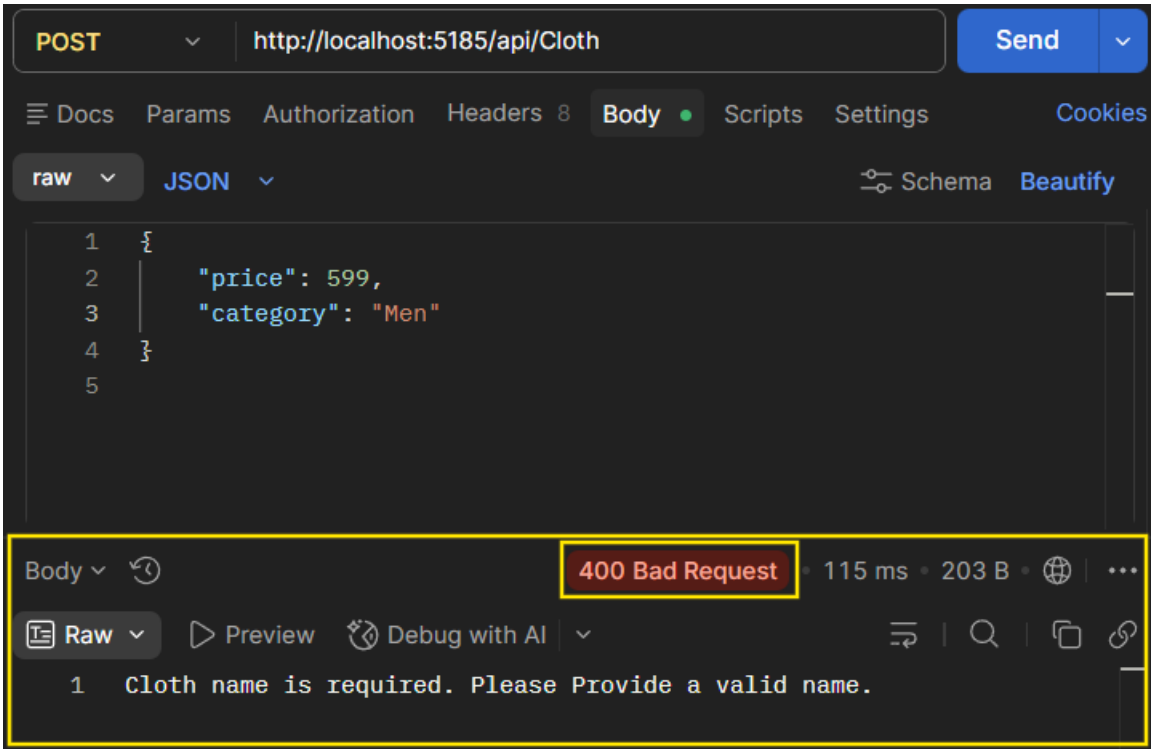
Request: POST http://localhost:5185/api/Cloth

Request Body:

Content-Type: application/json

```
{
    "price": 599,
    "category": "Men"
}
```

Response (400 Bad Request):



Test 2: 400 Bad Request – Invalid Price

Request: POST http://localhost:5185/api/Cloth

Request Body:

Content-Type: application/json

{

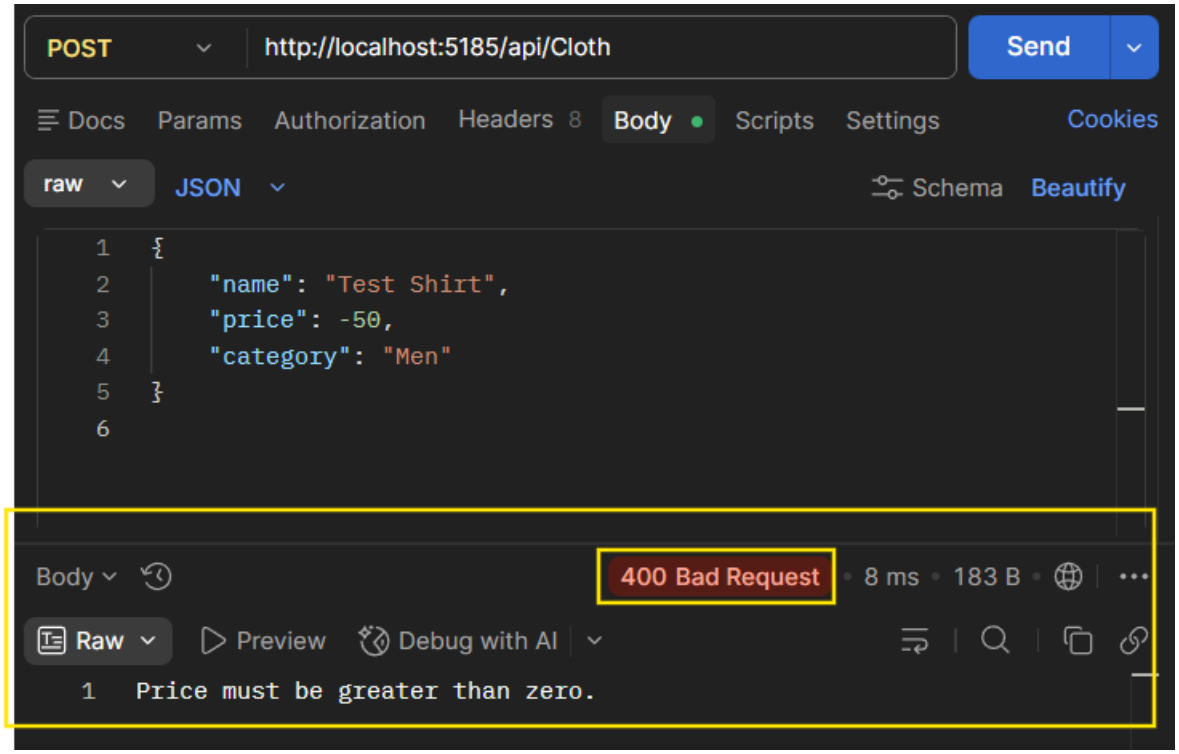
"name": "Test Shirt",

"price": -50,

"category": "Men"

}

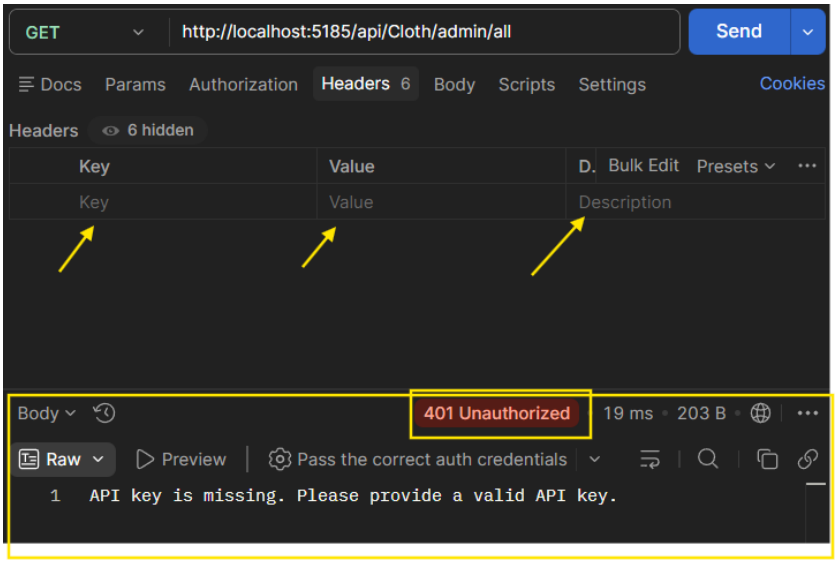
Response (400 Bad Request):



Test 3: 401 Bad Unauthorized – Missing API Key

Request: **GET http://localhost:5185/api/Cloth/admin/all**

Response (401 Unauthorized):



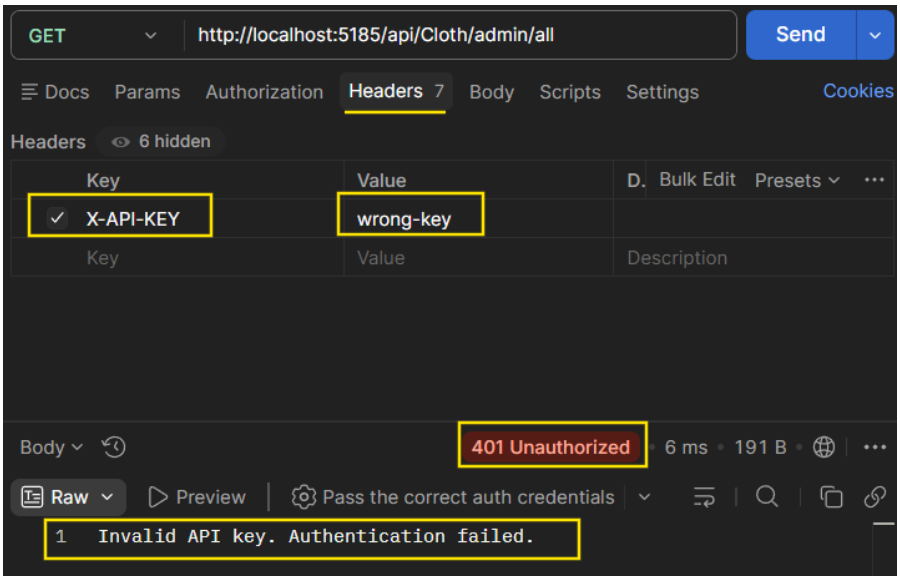
Test 4: 401 Bad Unauthorized – Invalid API Key

Request: **GET http://localhost:5185/api/Cloth/admin/all**

From Request Header:

X-API-Key: wrong-key

Response (401 Unauthorized):



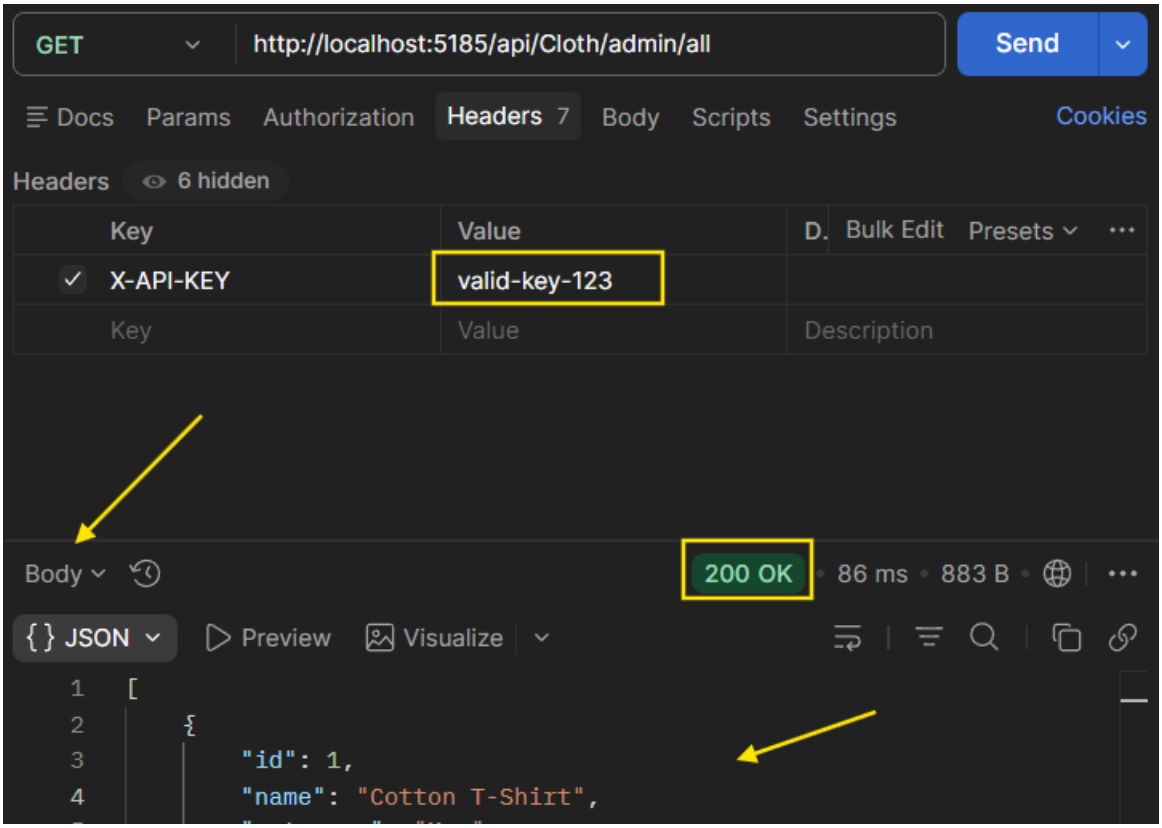
Test 5: 401 Unauthorized – Valid API Key (Success)

Request: GET http://localhost:5185/api/Cloth/admin/all

From Request Header (Keys):

X-API-Key: valid-key-123

Response (200 OK): Returns all 7 cloths



Test 6: 403 Forbidden – Valid API Key but No Admin Role

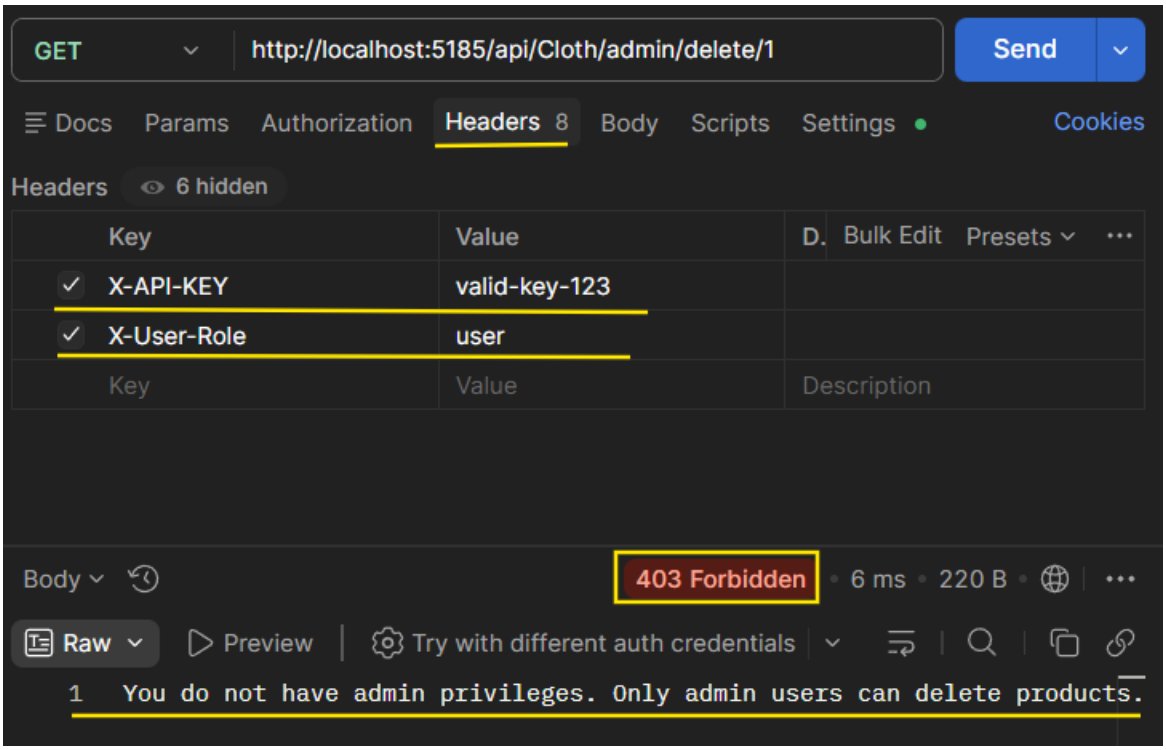
Request: GET http://localhost:5185/api/Cloth/admin/delete/1

From Request Header (Keys):

X-API-Key: valid-key-123

X-User-Role: user

Response (403 Forbidden):



Test 7: 403 Forbidden – Valid API Key + Admin Role (Success)

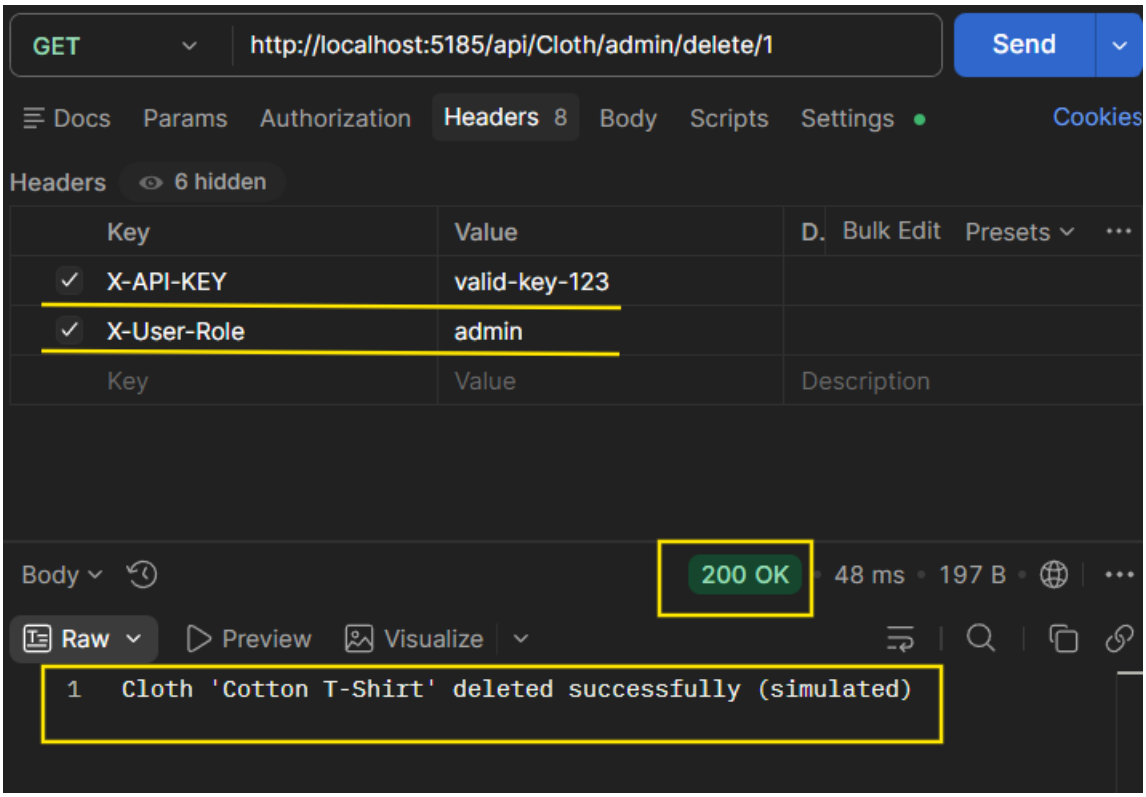
Request: **GET http://localhost:5185/api/Cloth/admin/delete/1**

From Request Header (Keys):

X-API-Key: valid-key-123

X-User-Role: admin

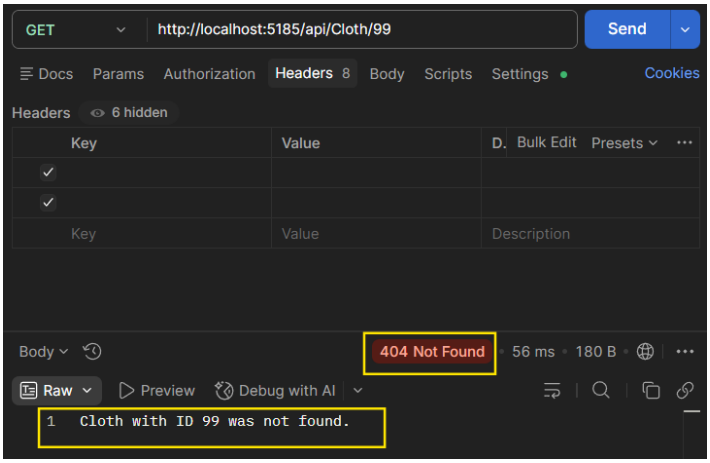
Response (200 OK):



Test 8: 404 Not Found – Invalid ID

Request: GET http://localhost:5185/api/Cloth/99

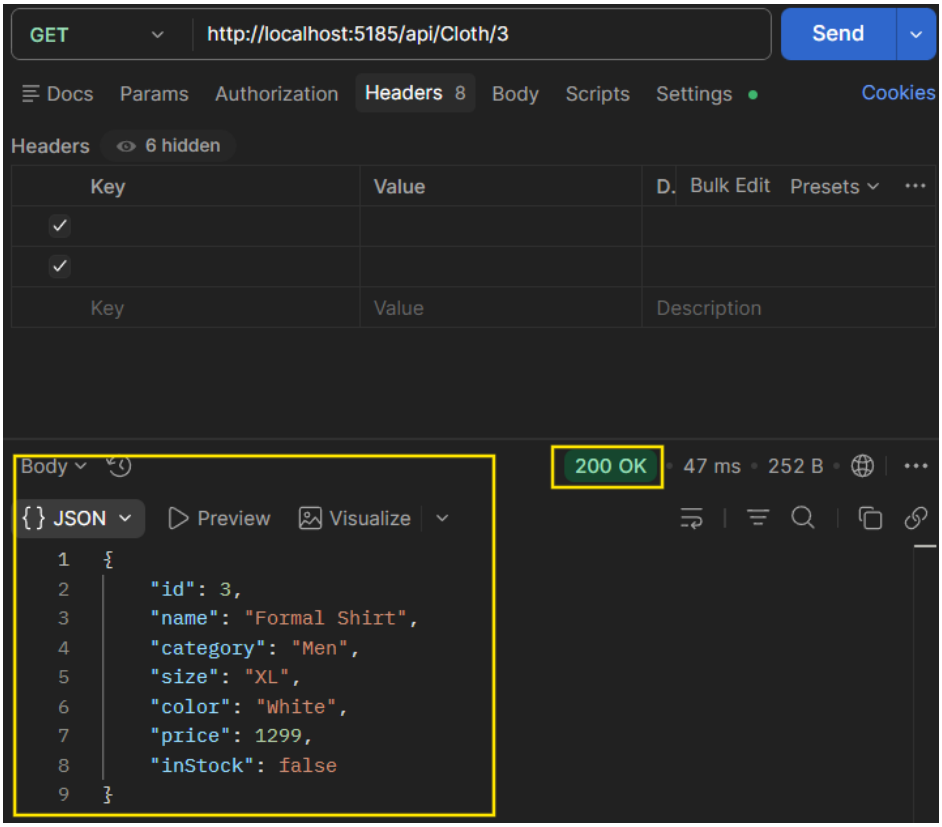
Response (404 Not Found):



Test 9: 404 Not Found – Valid Id (Success)

Request: GET http://localhost:5185/api/Cloth/3

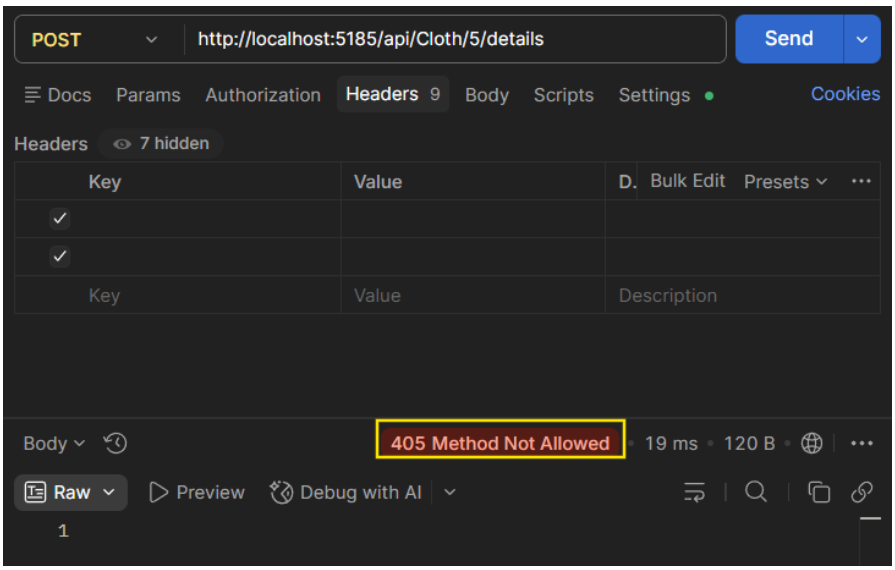
Response (200 OK):



Test 10: 405 Method Not Allowed – Wrong HTTP Method

Request: GET http://localhost:5185/api/Cloth/5/details

Response (405 Method Not Allowed):



Note: This is returned automatically by the framework because the endpoint only has **[HttpGet]** attribute.

Key Takeaways – 4xx Status Codes

Status Codes	Helper Method	When to Use
400	BadRequest()	Invalid input, missing required field, validation failed
401	Unauthorized()	Missing or invalid API key/token (authentication failed)
403	Forbid()	Valid authentication but insufficient permissions (authorization failed)
404	NotFound()	Resource does not exist at the requested URL
405	Automatic	HTTP method not supported by endpoint

Important Note About Forbid()

What We Tried	What Happened	Why
return Forbid()	Shows 500 Internal Server Error	**Forbid()** requires authentication middleware to be properly configured
Alternative approach	Using **return StatusCode(403, "message")**	Works immediately without additional configuration